

Persistencia y Datos

Relacional/OLTP · OLAP/ETL · NoSQL · CAP · replicación · sharding
· Map-Reduce · localidad

Ingeniería de Software II · ITBA · 2026-1C · Repaso conceptual

RESUMEN

La **persistencia** es la característica de los datos que sobreviven al programa que los genera (source: persistencia.md). Elegir cómo persistir es una **decisión arquitectónica de baja reversibilidad**: condiciona performance, escalabilidad, integridad y costo de evolución. Tres ejes de decisión: performance (acceso al medio + búsquedas), transformación (fidelidad al modelo de la app), tolerancia a fallos.

§1 Taxonomía de persistencia

DEFINICIÓN

Persistencia: los datos sobreviven al proceso que los crea. Necesaria porque la RAM es **volátil** (se pierde al apagar) y **acotada** (mucho más cara por byte que el disco) (source: persistencia.md).

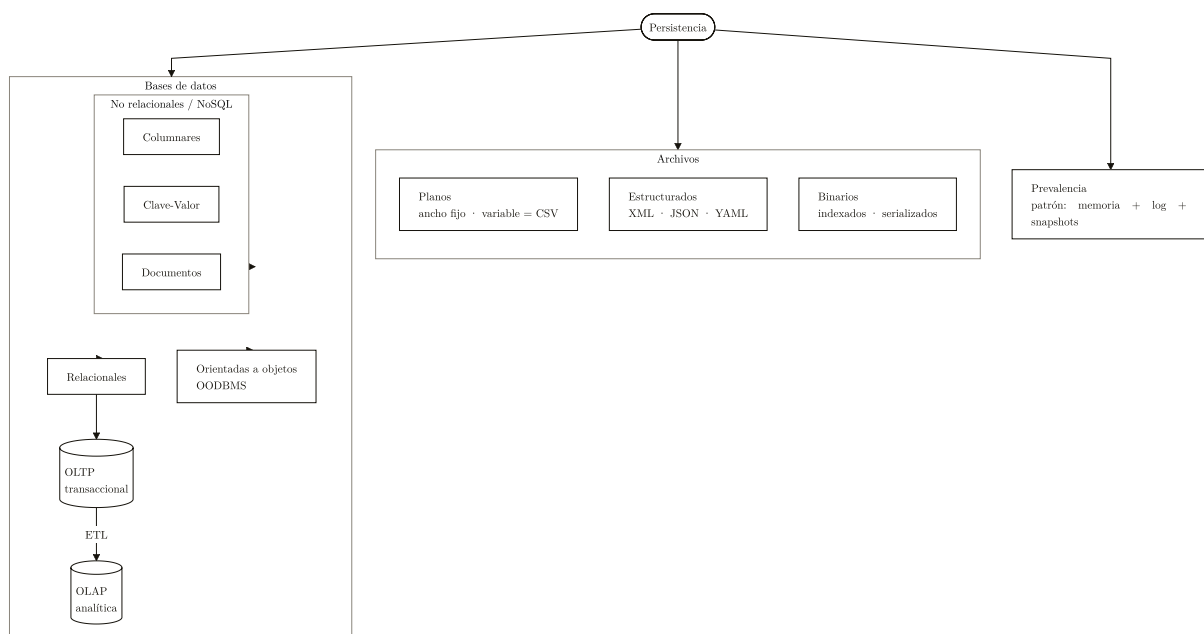


FIGURA 1. *Taxonomía de persistencia: archivos, el patrón de prevalencia y las familias de bases de datos. La OLAP siempre se deriva de la OLTP vía ETL.*

REGLA DE ORO

Criterios de elección (cátedra): (1) **Performance** — acceso al medio y costo de búsquedas; (2) **Transformación** — ¿cuán fiel es el almacenamiento al modelo de la app?; (3) **Tolerancia a fallos** — ACID, replicación, snapshots, WAL. Sumado al contexto del Architecture Business Cycle: presupuesto, equipo, regulación, volumen (source: persistencia.md).

CUÁNDO NO • CUIDADO

Archivo vs BD: archivo plano para configuración, logs, datasets read-only de tamaño moderado. BD cuando hay **conurrencia** o reglas de negocio sobre los datos. Una mala decisión de persistencia es **costosísima de revertir**: migraciones de meses, downtime, doble-escritura (source: persistencia.md).

§2 Prevalencia (in-memory + WAL + snapshots)

DEFINICIÓN

Prevalencia: *patrón* de persistencia (no tecnología) donde el estado completo vive en memoria como objetos nativos del lenguaje, y la durabilidad se garantiza con dos mecanismos (source: prevalencia.md):

- **Command log (write-ahead log):** cada operación que muta el estado se serializa a disco *antes* de aplicarse en memoria.
- **Snapshots periódicos:** se serializa todo el estado. Al arrancar: restaurar último snapshot + reaplicar el log posterior.

Implementaciones citadas: *Prevayler* (Java, Wuestefeld 2001), *Madeleine* (Ruby), *Bamboo* (.NET). *Redis con AOF+RDB es esencialmente prevalencia con otro nombre* (source: prevalencia.md).

CUÁNDO SÍ • VENTAJAS

Cuándo sí: dataset **acotado** que cabe en RAM y se lee mucho (configuración, reglas, índices chicos); prototipos/MVPs; trading o juegos donde la latencia importa más que el volumen. Sin impedancia objeto-relacional, performance "muchísimo más alta" (no hay traducción ni round-trip).

CUÁNDO NO • CUIDADO

Cuándo no: la app entera debe caber en memoria — no escala si el dataset crece indefinidamente (logs, eventos). Uniproseso por diseño: datos no compartibles entre apps. Interoperabilidad pobre (serialización del lenguaje), sin queries ad-hoc.

§3 Relacionales · OLTP · ACID · normalización

DEFINICIÓN

RDBMS: modelo relacional de **Codd (1970)** — datos en tablas (relaciones de filas y columnas), vínculos por foreign keys, consultas declarativas sobre **álgebra relacional y teoría de conjuntos** (SQL). **OLTP** = Online Transaction Processing: muchas transacciones cortas, alta concurrencia, ACID (source: bases-de-datos-relacionales.md).

CUÁNDO SÍ · VENTAJAS

Por qué son el default de la cátedra: (1) **independencia física** (storage abstracto del modelo lógico); (2) **acceso concurrente seguro** (locks, MVCC, niveles de aislamiento); (3) **recuperabilidad** (WAL, PITR, replicación); (4) **integridad** (constraints, ACID); (5) performance (índices, query planner, caches). **Normalización** (1NF→BCNF) elimina redundancia (source: bases-de-datos-relacionales.md).

CUÁNDO NO · CUIDADO

Cómputo en la base (stored procedures) — **en desuso:** la cátedra lo **desaconseja** explícitamente: rompe CI/CD, dificulta versionado, rollback y progressive rollouts (source: bases-de-datos-relacionales.md).

El quiebre 2007-2008 y los workarounds

Las redes sociales empujaron a los RDBMS fuera de su zona de confort: concurrencia inédita, DDL que bloquea tablas, volúmenes que no entran en RAM. Workarounds creativos **antes** de migrar (source: bases-de-datos-relacionales.md):

- **Sharding** — partir la base en N copias estructuralmente iguales con datos disjuntos.
- **Tablas-como-columnas** — una tabla por atributo (`id+valor`); agrega columnas sin downtime, a costa de joins múltiples.
- **JSON blobs** — FriendFeed (2009) sobre MySQL: tabla (`uuid`, `json_blob`) + tablas auxiliares como índices secundarios.

FUENTE

"¿Estamos usando la base de datos como tal?" — si los hacks degradan el RDBMS (lógica ad-hoc, pérdida de consistencia y transaccionalidad), el ahorro del cambio se evaporó (source: bases-de-datos-relacionales.md).

§4 OLAP y ETL

DEFINICIÓN

OLAP — **Online Analytical Processing:** BD relacional diseñada **exclusivamente para reporting analítico** sobre grandes volúmenes históricos. **Siempre se popula a partir de una OLTP** — es un derivado, no la fuente de verdad (source: olap-etl.md).

TABLA 1. OLTP vs OLAP

Eje	OLTP	OLAP
Carga	Muchas writes cortas + lookups por PK	Pocas queries pesadas con agregaciones
Esquema	Normalizado (3NF)	Desnormalizado (star/snowflake), evita joins
Mutabilidad	INSERT/UPDATE/DELETE	Append-only
Agregaciones	En query-time	Precalculadas al cargar
Latencia	< 10 ms	Segundos a minutos
Indexación	B-tree por PK + FKs	Bitmap, columnar, cubos

ETL — Extract, Transform, Load: proceso **batch** OLTP→OLAP. *Extract* (dump, CDC, incremental) → *Transform* (limpia, denormaliza, agrega — aquí vive la lógica analítica) → *Load* (bulk insert append-only). **ELT** invierte el orden (E→L→T): carga en bruto al warehouse columnar (BigQuery, Snowflake, Redshift) y transforma con SQL adentro; aprovecha cómputo cloud elástico, caso emblemático dbt (source: olap-etl.md).

CUÁNDO NO • CUIDADO

Nunca: reporting pesado sobre la OLTP viva — normalización y locks penalizan los queries, las agregaciones bloquean tablas vivas. Separar en cuanto el reporting degrade la latencia OLTP o el histórico supere ~10× el dataset vivo (source: olap-etl.md).

§5 Bases de datos de objetos (OODBMS)

DEFINICIÓN

OODBMS: almacenan objetos complejos directamente — preservan **herencia**, **polimorfismo**, **asociaciones bidireccionales** e **identidad de objeto**. No hay tablas ni filas: un grafo de objetos persistente. Soportan muchos-a-muchos sin tablas intermedias (source: bases-de-datos-objetos.md).

CUÁNDO NO • CUIDADO

Por qué casi nunca se eligen: perdieron la guerra contra los RDBMS por razones **extra-técnicas** — ecosistema (50 años de tooling/DBAs/drivers), estándar SQL (ODMG/OQL nunca tuvo tracción), acoplamiento al lenguaje, migración de esquema invasiva. El problema que resolvían (la impedancia) se mitigó con **ORMs sobre RDBMS**. Respuesta por defecto: RDBMS + ORM (source: bases-de-datos-objetos.md).

Raros casos válidos: lenguajes de objetos puros (*Smalltalk*), jerarquías de herencia profundas donde el ORM es pesadilla. Productos: *Db4o*, *ObjectDB*, *Versant*, *GemStone/S*.

§6 ORM e impedancia objeto-relacional

FUENTE

"El choque entre el modelo de objetos vs el almacenamiento de tuplas" (source: orm-impedancia-objeto-relacional.md, slide Impedancia Objeto-Relacional).

La **impedancia objeto-relacional** es la fricción estructural entre el grafo de objetos (identidad por referencia, herencia, polimorfismo, asociaciones bidireccionales) y las tablas planas (identidad por PK, sin herencia, FKs unidireccionales). Puntos de choque: inflexibilidad de las tablas, falta de herencia, **identidad vs PK**, datos vs comportamiento, FKs vs bidireccionalidad (source: orm-impedancia-objeto-relacional.md).

CUÁNDO SÍ • VENTAJAS

ORM — ventajas: productividad (menos boilerplate), portabilidad de motor, encapsulamiento (dominio sin SQL). Mapea clases↔tablas, atributos↔columnas, automatiza CRUD, gestiona identity map, lazy loading, dirty tracking.

CUÁNDO NO • CUIDADO

ORM — costos: SQL subóptimo y problema **N+1**; complejidad (capa extra, bugs en dos modelos); abstracción que **falla bajo presión**: para queries complejas hay que escapar a SQL nativo — "You can't use the ORM for everything".

Patrones canónicos (Fowler, PoEAA parte III)

Active Record

La clase de dominio *es* la mapeada y conoce su persistencia (`pedido.save()`). Simple, acoplado. Ej: ActiveRecord (Rails), Eloquent (Laravel).

Data Mapper

Separa dominio del mecanismo de persistencia: dominio puro + mapper aparte. Ej: Hibernate, JPA, Doctrine, SQLAlchemy.

Repository

Colección-como-fachada sobre el Data Mapper; oculta queries.

Unit of Work

Registra los cambios de una sesión y los commitea en **una transacción**.

TRADE-OFF

Active Record ↔ Data Mapper: AR en apps CRUD-heavy con dominio simple; DM en sistemas grandes, dominio rico, testing intensivo. Regla práctica: **ORM por default para el 80%**, SQL escapado para queries complejas o performance crítica (source: orm-impedancia-objeto-relacional.md).

§7 Replicación de BD

Tres motivaciones distintas: **escalabilidad de lectura**, **alta disponibilidad** (failover) y **localidad geográfica** (source: replicacion-bd.md).

TABLA 2. Topologías de replicación

Modelo	Cómo	Ventaja	Costo
Primario–Secundario (P-S)	1 primario acepta writes; N réplicas read-only reciben cambios (async via WAL streaming); failover si cae	Modelo más soportado ; operacionalmente simple; ideal read-heavy ($\geq 80/20$)	Primario = cuello de botella de escritura; lag de replicación; failover no instantáneo
Primario–Primario (multi-master)	N nodos aceptan reads y writes; propagación bidireccional	Escala escrituras; tolera caída de cualquier nodo sin failover	Conflictos de concurrencia (autoincrementales); fibra + co-ubicación + kernel modules; overhead $O(N^2)$ vs capacidad $O(N)$ — tope ~8-16 nodos

TRADE-OFF

Sincrónica ↔ asincrónica (eje ortogonal a la topología): **sync** = el commit espera confirmación de réplicas → consistencia, a costa de latencia (CP-priorizante). **async** = commit retorna al escribir en el primario → mejor latencia, riesgo de pérdida si el primario muere antes de propagar (AP-priorizante). **Semi-sync** = espera $K < N$ réplicas (compromiso popular). Default cátedra: **P-S asincrónica** (source: replicacion-bd.md).

§8 Teorema CAP

DEFINICIÓN

CAP (Brewer 2000; Gilbert & Lynch 2002): ante una **partición de red**, un sistema distribuido debe elegir entre **Consistencia** y **Disponibilidad** — no puede ofrecer ambas. **C** onsistency: toda lectura ve la última escritura. **A** vailability: todo pedido a un nodo sano recibe respuesta. **P** artition tolerance: opera aunque la red se rompa (source: teorema-cap.md).

REGLA DE ORO

El "elegí 2 de 3" es engañoso: **P no es opcional** (toda red real particiona). La elección real es **C vs A durante la partición**. **CP**: el lado minoritario rechaza ops para no divergir (pierdo A) — HBase, MongoDB majority, etcd, Zookeeper. **AP**: todos siguen aceptando, reconcilio al reunificar (pierdo C momentánea) — Cassandra, DynamoDB, CouchDB, Riak. **CA** sólo sin partición (RDBMS single-node) (source: teorema-cap.md).

PACELC (Abadi 2010) refina: bajo Partición elegís A/C; **Else** (operación normal) elegís Latency/Consistency. Cassandra=PA/EL; Spanner=PC/EC. RDBMS clásico = C-priorizante; NoSQL frecuentemente AP-priorizante con eventual consistency (last-write-wins, vector clocks, CRDTs).

EJEMPLO

Decisión por dominio: banca/finanzas → **CP** (mejor rechazar que aceptar saldos divergentes); carrito e-commerce → **AP** (Amazon: siempre permitir agregar, reconciliar después); feed social → **AP** (un 503 es peor que ver datos 30s viejos); coordinación/locks → **CP** (la consistencia es la razón de existir) (source: teorema-cap.md).

§9 Sharding

DEFINICIÓN

Sharding (particionado horizontal): "romper la base en N copias estructuralmente iguales, pero con data disociada". Cada shard usa el **mismo esquema**, un **subconjunto disjunto** de datos, y sus propias réplicas. Cada shard maneja menos volumen → índices entran en RAM; carga distribuida en N (source: sharding.md).

TABLA 3. Estrategias de partición (elección de shard key)

Estrategia	Cómo	Trade-off
Rango (range)	cada shard cubre un rango de la clave	Soporta queries de rango; riesgo de hot shards si está sesgado
Hash	$\text{shard} = \text{hash}(\text{key}) \bmod N$	Distribución uniforme; pierde locality de rango (query de rango toca todos)
Lookup / directorio	tabla externa mapea claves a shards	Flexible (permite rebalanceo); un componente más
Geográfico	shard por región	Soberanía de datos; latencia regional
Temporal	shard por ventana de tiempo	Twitter para tweets: locality temporal — la mayoría pide los recientes (1 shard activo)

CUÁNDO NO • CUIDADO

Desafío central: (1) la app debe saber **a qué shard ir** → aplicá el **principio de localidad**: "qué datos se acceden juntos, se guardan juntos"; particioná por la dimensión del query dominante. (2) **No hay FKs, transacciones ni integridad cross-shard** — la provee la app: 2PC, retry-until-success (idempotencia), last-write-wins, sagas/compensaciones. Esto reexpone los trade-offs de CAP que el RDBMS ocultaba (source: sharding.md).

REGLA DE ORO

No hay shard key universal: Twitter probó partition-by-id (timeline toca N shards) y partition-by-user_id (buscar tweet por id toca N) antes de elegir **partition-by-time**, privilegiando el caso común aceptando que los raros sean más lentos. La elección de shard key es de las decisiones más arquitectónicas que existen (source: sharding.md, principio-de-localidad.md).

§10 Bases no relacionales (NoSQL)

DEFINICIÓN

Bases no relacionales: motores que **productizan** los workarounds del RDBMS (sharding, blobs, columnas) y los abstraen al cliente. Definieron protocolos propios → se llamaron "NO-SQL"; hoy casi todas soportan algún SQL, así que el nombre correcto es "**no relacionales**". Rasgos: escalabilidad y distribución *por diseño*, volúmenes TB/PB, esquemas flexibles, **ACID opcional**. Mayoría AP-priorizante (source: bases-no-relacionales.md).

TABLA 4. Las tres familias NoSQL

Familia	Modelo	Fuerte en	Débil en	Productos
Columnar (wide-column)	datos por columnna, no por registro	proyecciones sobre pocas columnas; multi-valor nativo	queries que tocan muchas columnas (reconstruir fila)	BigTable, Cassandra, ClickHouse
Clave-Valor	key → value (opaco o estructurado)	flexibilidad máxima; latencia baja	no tipa; búsquedas multi-campo (AND/OR) complicadas	Redis, DynamoDB, Memcached
Documentos	documentos arbitrarios (JSON/BSON)	estructuras variables; REST→JSON; sharding nativo	proyecciones/joins más difíciles que SQL	MongoDB, ElasticSearch, CouchDB

EJEMPLO

Decisión por caso de uso: cache/sesiones/leaderboards → Redis (KV); catálogo de esquema variable/CMS → MongoDB (doc); logs/métricas/time-series → Cassandra/ClickHouse (columnar); full-text/autocompletado → ElasticSearch; transaccional con integridad referencial → **NO NoSQL, usar RDBMS** (source: bases-no-relacionales.md).

Polyglot persistence: los sistemas grandes combinan motores — Postgres core, Redis cache, ElasticSearch búsqueda, warehouse analítica. Costo: complejidad operativa + sincronización entre stores (CDC, event-driven).

§11 Map-Reduce

DEFINICIÓN

Map-Reduce: patrón para procesar grandes volúmenes **en paralelo** distribuyendo el cómputo (paradigma funcional; Google 2004). Tres fases: **Map** (cada nodo aplica f a su porción → pares clave-valor) → **Shuffle** (agrupa por clave, redistribuye) → **Reduce** (agrega los valores de cada clave) (source: map-reduce.md).

CUÁNDO SÍ • VENTAJAS

Ventajas: escalabilidad sobre máquinas commodity; tolerante a fallos (workers reagendados, datos replicados); modelo simple (sólo escribís map+reduce); independiente del lenguaje.

CUÁNDO NO • CUIDADO

Limitaciones: joins/proyecciones requieren varias rondas; **latencia alta** por el shuffle (I/O masivo) — **no sirve para queries online**; iterativo (ML, grafos) ineficiente (relee de disco cada vuelta).

*Evolución: Google MapReduce (2004) → Hadoop (2006) → **Spark** (2014, en memoria con DAGs, hoy default) → Flink/Beam. El patrón sigue vigente como mecanismo interno de warehouses y BDs columnares. Por eso la cátedra lo trata en Persistencia: la frontera almacenamiento↔procesamiento se difumina cuando los datos no entran en una máquina (source: map-reduce.md).*

§12 Localidad, memory hierarchy y el mantra de Kallen

Principio de localidad

DEFINICIÓN

Principio de localidad: "qué datos se acceden juntos, se deben guardar juntos."
Heredado de caches/memorias, aplicado a sharding y placement. **Espacial:** si accedo a i , pronto a $i+1$ (clustered indexes, shard por user_id, columnar). **Temporal:** si accedo en t , vuelvo pronto (caches, shard por tiempo, TTL) (source: principio-de-localidad.md).

CUÁNDO NO • CUIDADO

Antipatrón — ignorar localidad: cada query toca todos los shards (**scatter-gather**) → el sharding no compra escalabilidad; cache hit rate bajísimo. Cura: alinear shard key al query dominante. *Excepción:* a veces conviene **violar localidad** con hash uniforme para evitar hot shards cuando la distribución está muy sesgada — trade-off localidad vs balanceo (source: principio-de-localidad.md).

Memory hierarchy — "disk is for writes only"

DEFINICIÓN

Memory hierarchy: los medios forman una jerarquía con trade-off velocidad/capacidad/costo. Cada escalón = factor $10\times$ - $100\times$ en latencia. Registros $<1\text{ns}$, L1 $\sim 1\text{ns}$, RAM $\sim 100\text{ns}$, SSD $\sim 100\mu\text{s}$, HDD $\sim 10\text{ms}$, WAN 10^7 - 10^8ns . Diseñar = decidir conscientemente **en qué capa vive cada dato** según cómo se accede (source: memory-hierarchy.md).

FUENTE

"All data for real-time queries **MUST** be in memory. **Disk is for writes only.**" — Kallen (source: memory-hierarchy.md). En sistemas con SLA de milisegundos, el path crítico no toca disco: el disco sólo es medio durable (WAL, snapshots). Las lecturas vienen de RAM (Memcached, Redis).

EJEMPLO

Twitter — **fanout a memoria, no a disco**: fanout a disco = "ridonculous number of IOPS" (inviabile); fanout a Memcached = viable. Las timelines son **non-durable**: si Memcached cae se reconstruyen mergeando los timelines de los seguidos. Costo_reconstrucción $\times$ prob_falla < costo de IOPS (source: memory-hierarchy.md).

Partition / Replicate / Index

FUENTE

"Scalability solutions aren't magic. They involve **partitioning, indexing, and replication.**" — Kallen (source: partition-replicate-index.md). Tesis central: detrás de cada sistema escalable hay alguna combinación de estos tres ladrillos, ningún otro.

TABLA 5. Los tres mecanismos son complementarios

Mecanismo	Resuelve	No resuelve
Partition (sharding)	volumen + paralelismo	disponibilidad ante caída de nodo
Replicate	disponibilidad + throughput de lectura	volumen (todos tienen los mismos datos)
Index	velocidad de queries específicos	volumen ni disponibilidad

REGLA DE ORO

Heurística de orden: partition first (es lo que más escala); replicar cuando no se tolera caer un nodo o las lecturas exceden un nodo; índice secundario cuando una query frecuente requiere scatter-gather. Los índices son **materialización selectiva**: pagás escrituras y storage para abaratar las lecturas que importan (source: partition-replicate-index.md).

§13 Off-line vs online computation

DEFINICIÓN

¿Cuándo se hace el cómputo? es tan arquitectónico como qué se computa. **Online** (query-time): simple, siempre fresco, caro si es complejo/frecuente. **Off-line** (precomputado/materializado): lectura barata, escritura cara. **Híbrido**: materializo + recomputo on cache miss (source: off-line-vs-online-computation.md).

FUENTE

"The answer to some problems can be pre-computed if the amount of work is **bounded** and the query pattern is **very limited**." — Kallen. A la inversa: "It is not possible to pre-compute set algebra queries" (cómputo ilimitado → no se precomputa) (source: off-line-vs-online-computation.md).

TABLA 6. Read amplification vs Write amplification

Estrategia	Read	Write	Cuándo
Online (join en query-time)	Alto (joins, scatter-gather)	Bajo (un INSERT)	Reads ≪ Writes; queries ad-hoc/impredecibles
Off-line (materialización + fanout)	Bajo (lookup directo)	Alto (escribir N copias)	Reads ≫ Writes; queries predecibles; fanout bounded
Híbrido (cache + recompute)	Bajo si hit, alto si miss	Bajo + invalidación	Reads ≫ Writes con patrones variables

EJEMPLO

Twitter timelines (off-line): timeline = secuencia de tweet ids materializada en Memcached. Al tweetear, un proceso **fanout** escribe el id en el timeline de cada seguidor (~600:1 promedio); read es O(1) (GET timeline:user_id), write es O(seguidores) → **1.2M deliveries/seg**. Generaliza a vistas materializadas, OLAP cubes, índices invertidos (Lucene), CDNs (source: off-line-vs-online-computation.md).

EN EL PARCIAL

Persistencia es el tema más grande: esperá una pregunta de **decisión** (¿qué motor para este dominio? ¿CP o AP? ¿qué shard key?). Una buena respuesta: nombra el **criterio** (patrón de query, ratio read/write, SLA, integridad), cita el trade-off (CAP, read vs write amplification, localidad vs balanceo) y dice **cuándo NO** aplica. Errores que matan: confundir replicación (misma data, escala lecturas/HA) con sharding (data disjunta, escala volumen/writes); usar OLTP para reporting; decir "elegí 2 de 3 de CAP" sin aclarar que P es obligatorio.